E-commerce Test Strategy Document

1. Introduction

This document defines the comprehensive test strategy for an E-commerce platform covering Authentication, Cart, Checkout, and Payment modules.

2. Scope

In Scope: Login/Signup, OTP, Cart, Checkout, Payment Success/Failure flows.

Out of Scope: Inventory, Admin modules (assumed).

3. Test Approach

Testing will include Functional, Integration, Regression, UAT, API, and End-to-End flows.

4. Risk-Based Testing Strategy

High Risk: Payment failures, duplicate transactions, OTP delays.

Medium Risk: Cart persistence, session timeouts.

Low Risk: UI validation.

Risk Matrix: Impact vs Probability used to prioritize execution.

5. Test Types

Functional Testing, API Testing, Integration Testing, Regression Testing, UAT Support.

Performance Testing: 5000 concurrent users, 3x spike.

Security Testing: Payment data, authentication vulnerabilities.

6. Test Environment

Dev, QA, UAT, Pre-prod, Prod environments.

Assumed AWS cloud, microservices architecture, Jenkins CI/CD.

7. Test Data Strategy
Test users, payment scenarios, OTP simulations, masked production-like data.

8. Automation Strategy
UI Automation: Selenium WebDriver.

API Automation: RestAssured.

CI/CD: Jenkins pipelines integrated with test suites.

Goal: 70% regression automation coverage.

9. Defect Management
Tools: Jira (assumed).

Lifecycle: New → Assigned → In Progress → Fixed → Retest → Closed.

10. Defect Metrics
Defect Density: ~0.5 defects/test case.

Defect Leakage: <5%.

Defect Removal Efficiency: ~95%.

11. Entry and Exit Criteria
Entry: Stable build, environment ready.

Exit: Critical defects closed, acceptable defect leakage.

12. Deliverables
Test Plan, Test Cases, Automation Scripts, Test Reports, Defect Reports.

13. Reporting & Dashboards
Daily status reports, defect trends, test execution reports.

14. Roles & Responsibilities
QA Lead, Test Engineers, Automation Engineers, DevOps.

📊 Detailed Risk Matrix (Impact vs Probability with Scoring)

Risk Area	Impact (1–5)	Probability (1–5)	Risk Score (Impact × Probability)	Priority
Payment failures	5	4	20	High
Duplicate transactions	5	3	15	High
OTP delays	4	3	12	Medium
Cart persistence issues	3	3	9	Medium
Session timeouts	3	2	6	Low
UI validation defects	2	2	4	Low

Sample QA Dashboards (Excel + Charts)

- **Defect Trend Chart**: Line chart showing defects opened vs closed per sprint.
- **Test Execution Dashboard**: Pie chart showing % of test cases passed, failed, blocked.
- **Automation Coverage**: Bar chart showing regression automation coverage vs manual.
- **Defect Density Heatmap**: Module-wise defect density visualization.

🧩 Microservices Test Architecture Diagram

- **Frontend Layer** → Web/Mobile UI.
- **API Gateway** → Routes requests to microservices.
- **Microservices** → Cart, Checkout, Payment, Authentication.

- **Database Layer** → Separate DB per service.
- **Monitoring Layer** → Logs, alerts, dashboards.
- **Testing Hooks** → API mocks, service virtualization, automated regression packs.

🔄 **CI/CD Pipeline Workflow (Visual)**

1. **Code Commit** → GitHub/GitLab.
2. **Build** → Jenkins CI triggers build.
3. **Unit Tests** → Executed automatically.
4. **Integration Tests** → API + service-level tests.
5. **Regression Automation** → Selenium + RestAssured suites.
6. **Deployment** → QA → UAT → Pre-prod → Prod.
7. **Monitoring & Feedback** → Defect dashboards, alerts.

📋 **Detailed Test Case Samples & RTM**

Sample Test Case (Checkout Module)

- **ID**: TC-CHK-001
- **Title**: Verify successful checkout with valid card.
- **Precondition**: User logged in, cart has items.
- **Steps**:
    1. Navigate to checkout.
    2. Enter valid card details.
    3. Submit payment.
- **Expected Result**: Payment success message, order ID generated.
- **Priority**: High.

Requirements Traceability Matrix (RTM)

Requirement ID	Requirement Description	Test Case IDs	Status
REQ-001	User login with OTP	TC-LOGIN-001,002	Covered
REQ-002	Cart persistence across sessions	TC-CART-005,006	Covered
REQ-003	Payment success/failure handling	TC-CHK-001,002,003	Covered
REQ-004	Security validation for payments	TC-SEC-001,002	Covered